Auditoría del módulo de bitácora y eventos

Fecha: **15 de octubre de 2026**. Alcance: código backend y frontend del árbol de trabajo actual, incluidos sus cambios locales; referencia de HEAD: `9081d8a`. Instrucción aplicada: Agents.md. Esta entrega documenta hallazgos y acciones; no modifica lógica de negocio ni datos.

1. Resumen Ejecutivo

La bitácora funciona como una proyección editable de operaciones, pero no garantiza un historial íntegro e inmutable. Registra compras, ventas, pesajes, aplicaciones sanitarias, reproducción, camadas y traslados. Sin embargo, sus APIs permiten alterar eventos automáticos, algunas claves de actualización reemplazan sucesos distintos y varias operaciones guardan el dato principal antes de generar su evento.

Se identificaron **11 hallazgos: 6 P1 y 5 P2**. Los riesgos principales son:

- Un administrador puede modificar o borrar eventos automáticos de animales y camadas desde las APIs. Encargados y veterinarios también pueden modificar eventos automáticos de camadas.
- Los controladores reciben campos internos sin una lista permitida: origen, autor en edición, entidad relacionada y organización/finca. El aislamiento de consultas no protege por sí solo estos valores al escribir.
- Una salida de lote reemplaza la entrada de la misma pertenencia. En bandas porcinas, ingreso y retiro/finalización comparten identidad de evento; las actividades repetidas del mismo tipo también pueden reemplazarse.
- No hay validación temporal del evento contra nacimiento, muerte o fecha actual. Un registro reproductivo con solo fecha de parto estimada genera un supuesto diagnóstico en esa fecha futura.
- La bitácora de un animal trasladado sí consulta otras fincas autorizadas, pero excluye fincas inactivas y no muestra la finca de cada evento. No existe persistencia offline de la bitácora ni cola de notas manuales.

Hay controles útiles: autenticación en ambas APIs, permisos por rol, aislamiento organizacional de consultas, `createdAt`/`updatedAt`, autor asignado desde la sesión al crear notas, y traslados con historia independiente y transacción. Las operaciones comerciales revisadas también propagan sesiones a sus eventos. Estos controles no corrigen los problemas anteriores.

Alcance y mapa de implementación

Capa	Archivos y comportamiento comprobado
Esquemas	EventoAnimal, EventoCamada, HistorialFincaAnimal.
APIs	eventoAnimalRoutes, eventoCamadaRoutes, sus controlador animal y controlador camada. Montaje autenticado en app.js.
Escritura compartida	eventoAnimal-service, eventoCamada-service, aislamiento organizacional.
Fuentes automáticas	Controladores de animales, compras, ventas, pesajes, reproducción, tareas y camadas; servicios de sanidad, lotes, IATF, bandas porcinas y consolidación multi-finca. Se

Capa	Archivos y comportamiento comprobado
	contrastaron también rotaciones, alimentación e importación.
Interfaz real	Las dos líneas de tiempo están en Animales.js, no en componentes separados llamados BitacoraAnimal/HistorialEventos/FichaAnimal. Acceso desde ListaUsuario.js; llamadas en api.js.
Permisos y auditoría	permisosRoles.js, auditoria.js, auditoriaRoutes.js.
PWA	offlineStorage.js, offlineSync.js, vite.config.js.

`EventoAnimal` persiste `animal`, `tipoEvento`, `fecha`, `titulo`, `descripcion`, `moduloOrigen`, `referenciaId`, `creadoPor`, `metadata`, `organizacionId`, `fincaId` y timestamps. `EventoCamada` utiliza `camada` en lugar de `animal`. El autor es opcional en los esquemas; `metadata` es `Mixed`. Los índices por entidad/origen/referencia/tipo **no son únicos**.

`HistorialFincaAnimal` persiste animal, tipo de ingreso/traslado, finca origen/destino, fecha, peso, motivo, referencia al traslado y `registradoPor`, con timestamps y aislamiento por organización. No utiliza el filtro automático por finca de los otros dos modelos. No se encontró una API genérica para editar o borrar este historial.

Verificación realizada y límites

- Se ejecutó `npm.cmd test` en `backend`, equivalente a `npm test` en este entorno Windows: **214 pruebas aprobadas, 0 fallidas**.
- Se ejecutaron comprobaciones Node aisladas con funciones reales, validación real de esquemas y dobles en memoria para las llamadas de persistencia. Confirmaron: reemplazo entrada/salida de lote; borrado de un evento `Venta`; reenvío de campos internos en PUT de camada; aceptación de fechas 1900/2999 y organización/finca explícitas distintas al contexto; fecha estimada convertida en diagnóstico; desplazamiento de un día al presentar medianoche UTC en Costa Rica.
- Estas comprobaciones no conectaron a MongoDB. La prueba de DELETE/PUT verifica controladores, no una petición HTTP con autenticación. Los roles se contrastaron por separado contra rutas y configuración.
- No se ejecutaron ataques contra un servidor, pruebas de concurrencia MongoDB, navegación automatizada ni inspección de datos productivos. Los riesgos por carreras y fallos intermedios se identificaron por el código. Las 214 pruebas existentes no demuestran que los hallazgos estén cubiertos ni corregidos.

2. Catálogo de Tipos de Eventos

En la columna de mutación, **A** significa API de eventos de animal: administrador puede editar/borrar. **C** significa API de eventos de camada: administrador, encargado y veterinario pueden editar; solo administrador puede borrar. Esto aplica a eventos manuales y automáticos. La interfaz de la línea de tiempo no presenta botones de edición/borrado.

La API genérica acepta los tipos enumerados por cada esquema, aunque la interfaz manual solo crea `Observacion`. La existencia de un valor en el enum no demuestra que haya un productor automático.

Tipo de Evento	Origen (Manual / Automático)	Permite Edición/Borrado?	Datos que Registra
Observacion manual, animal	Manual, formulario de texto libre	A	Fecha actual del navegador, título fijo, descripción, autor de sesión; sin categorías clínicas ni adjuntos.
Observacion manual, camada	Manual, disponible por API; sin formulario dedicado en su timeline	C	Texto, fecha, autor y metadatos enviados.
Nacimiento, animal	Automático: Inventario y nacimiento desde Reproducción	A	Fecha de nacimiento, identificación, peso y genealogía según productor; referencia al animal o registro reproductivo.
Compra	Automático: Compras e Inventario	A	Fecha, proveedor, peso, precio, subtotal y monto asignado/ajustes en Compras; resumen de importe/fecha en Inventario.
Venta, animal/camada	Automático: Ventas; Inventario para estado vendido; ciertas Tareas	A / C	Venta real: comprador, peso, precio, monto, referencia comercial; camada añade cantidad. Tarea: finalización y observaciones, no comprobante comercial.
Muerte	Automático: Inventario; tarea de sacrificio de animal	A	Fecha, estado anterior y observaciones, o referencia a tarea completada.
Pesaje	Automático: Pesajes; ciertas Tareas	A	Pesaje real: fecha, peso y referencia a Pesaje. Tarea: descripción de finalización; no necesariamente una medición persistida.
Sanidad / Tratamiento, animal	Automático: aplicaciones sanitarias, cambios de estado sanitario y Tareas	A	Producto, dosis, vía, naturaleza, responsable, motivo, plan/tratamiento y aplicación; cambios de estado incluyen anterior/nuevo.
Monta	Automático: Reproducción, IA convencional, IATF, bandas porcinas y Tareas	A	Fecha y referencia reproductiva; IATF/bandas añaden campaña/paso y datos de semen según productor. El tipo no distingue por sí solo monta natural de IA.
Diagnostico de gestacion	Automático: IATF, bandas, cierre de ciclo no preñado y	A	Resultado/método y fecha cuando proviene de diagnóstico; el controlador general también usa una fecha estimada de parto como diagnóstico (BIT-06).

Tipo de Evento	Origen (Manual / Automático)	Permite Edición/Borrado?	Datos que Registra
	controlador reproductivo general		
Parto	Automático: Reproducción, alta de camada, bandas y ciertas Tareas	A	Fecha, crías/genealogía o referencia de camada con nacidos vivos/muertos; las tareas conservan su propia referencia.
Destete	Automático: Camadas, bandas y Tareas; rama condicional del controlador reproductivo general	A / C	Fecha, número destetado, peso promedio o referencia reproductiva/tarea, según flujo. No todas las fechas calculadas de destete generan un hecho ejecutado.
Cambio de lote	Automático: Lotes y restauración de pertenencia al revertir venta	A	Lote, código, pertenencia y acción entrada/movimiento/salida/restauración. Entrada/salida de una pertenencia colisionan.
Cambio de potrero	Admitido en esquema animal; no se encontró emisión automática a EventoAnimal	A si existe	El movimiento desde Lotes registra `CAMBIO_POTRERO` en EventoLote. Ese historial no se incorpora a la línea de tiempo individual.
Traslado de finca	Automático: Fincas	A	Dos eventos, salida e ingreso, con finca propia, fecha, referencia al traslado, IDs de origen/destino y peso.
Observacion automática	Automático: cambio de DIIO, inicio/salida/cancelación IATF, ingreso/retiro/cierre de banda, cierre/reapertura de ciclo	A	Identificación anterior/nueva, motivo y referencias de campaña, banda o ciclo según operación.
Camada registrada	Automático: Camadas	C	Fecha de nacimiento, madre, padre, código y nacidos totales/vivos/muertos/destino. También genera Parto en la madre.
Cambio de destino	Automático: edición de Camadas	C	Destino anterior/nuevo y distribución de crías; se crea sin referencia de upsert, conservando cambios sucesivos.
Cierre / Cancelacion de camada	Automático: Camadas	C	Fecha, motivo/observaciones y referencia de camada.

Tipo de Evento	Origen (Manual / Automático)	Permite Edición/Borrado?	Datos que Registra
Sacrificio de camada	Automático: tareas correspondientes	C	Fecha de finalización y observaciones/referencia de tarea.
Nacimiento / Sanidad / Tratamiento / Monta / Mortalidad / Observacion, camada	Valores admitidos por EventoCamada; emisión depende del flujo de Tareas/API	C	No se encontró un productor dedicado que genere `Mortalidad` por cada ajuste de muertos, ni integración de aplicación sanitaria real con EventoCamada. No debe asumirse esa cobertura por el enum.
Ingreso inicial / Traslado, historial de finca	Automático: HistorialFincaAnimal, separado de EventoAnimal	Sin CRUD genérico encontrado	Fincas origen/destino, fecha, peso, motivo, traslado y registradoPor. El ingreso inicial puede reconstruirse desde createdAt al primer traslado.

Fuentes representativas: inventario, compras, ventas, pesajes, aplicaciones sanitarias, reproducción, IATF, bandas, camadas y tareas.

Cobertura que no debe suponerse: los enum de origen incluyen Finanzas, Potreros y Alimentacion, pero no se encontró generación general de EventoAnimal desde sus operaciones. Alimentación y varios pasos de protocolos conservan historia propia de lote/campaña. La importación Excel tampoco llama al generador de eventos de inventario. Un alta importada no equivale a una reconstrucción de su vida productiva.

Notas y adjuntos: el formulario permite describir síntomas, accidentes o comportamiento como texto, sin tipos específicos ni datos estructurados. No existe selector de fecha histórica. No se encontraron campos, formulario ni endpoint de carga de adjuntos de bitácora. La fotografía principal del animal y `ArchivoMultimedia` no son adjuntos de una nota; el modelo multimedia no contempla ese uso. Guardar una URL arbitraria en `metadata` no constituye soporte de adjuntos.

3. Análisis de Integridad y Seguridad

BIT-01 — P1: modificación y borrado directo de eventos derivados

Evidencia: updateEvento/deleteEvento y actualizarEventoCamada/eliminarEventoCamada ejecutan actualización o borrado por ID sin comprobar `moduloOrigen`, `referenciaId` ni condición de nota manual. Las rutas imponen roles, pero no la integridad respecto del módulo fuente.

Caso verificable: con un administrador y un evento de venta de la finca activa, `DELETE /api/eventos-animal/:id` elimina el evento; no anula la venta ni impide el borrado por ser automático. Un veterinario o encargado puede cambiar por PUT el texto/fecha de un evento de venta de camada. El controlador de borrado animal devolvió 200 en la reproducción aislada con un evento `Ventas/Venta`.

Impacto: ficha y fuente comercial/médica pueden contradecirse. Ocultar botones en frontend no protege las rutas.

Acción: convertir las mutaciones públicas en operaciones exclusivas de notas manuales. Rechazar cambios directos de eventos derivados y tramitar rectificaciones desde el módulo fuente, preservando el hecho original y motivo de corrección/anulación.

BIT-02 — P1: campos internos modificables y relaciones sin validar

Evidencia: ambos POST expanden `req.body`; ambos PUT lo entregan completo a Mongoose. En POST se fuerza `creadoPor` desde la sesión, pero no se fija obligatoriamente origen Manual ni se descartan organización/finca/referencia. PUT permite cambiar también el autor. La API de camada no consulta Camada antes de crear un evento; las actualizaciones tampoco validan la entidad destino.

El plugin organizacional agrega organización/finca al **filtro** de consulta. Su hook de validación solo rellena valores ausentes y la protección de actualización se aplica a contexto de inserción de upserts, no a los valores enviados en estos PUT normales. Los campos de tenant no son inmutables en el esquema.

Reproducción aislada: bajo contexto de organización A/finca A, ambos modelos validaron documentos con organización B/finca B explícitas. El controlador PUT de camada reenvió sin filtrar `creadoPor`, `moduloOrigen`, `camada`, `organizacionId` y `fincaId`.

Impacto: un usuario con permiso de escritura puede fabricar un evento con apariencia automática, asociarlo a una entidad inexistente o incoherente y alterar su pertenencia. Esto no demuestra lectura arbitraria de otra organización: el filtro de consulta sí permanece aislado. El defecto comprobado está en la validación de escritura y la coherencia de referencias.

Acción: DTO de nota con campos permitidos; derivar autor/tenant/origen desde el servidor; prohibir operadores y cambios de campos internos; validar entidad, organización, finca y referencias. Probar tanto POST como PUT, incluidas actualizaciones con `$set`.

BIT-03 — P1: claves de upsert reemplazan sucesos diferentes

Evidencia: upsertEventoAnimal identifica un evento por animal + origen + referencia + tipo. registrarEventoLote usa la pertenencia como referencia para entrada y salida; `metadata.accion` no participa en la identidad. Se reprodujeron dos llamadas reales al generador con la misma pertenencia: quedó **un evento de salida**, con fecha y autor de salida, en lugar de dos hechos.

En bandas porcinas, ingreso, retiro y finalización/cancelación usan tipo `Observacion`, origen `Reproduccion` y referencia a la misma banda. Las actividades utilizan también la banda como referencia y dejan el paso solo en metadata (línea 242); dos pasos del mismo tipo pueden reemplazarse.

Impacto: la línea de tiempo pierde entradas o actividades anteriores aunque la pertenencia/protocolo conserve otros datos. No se afirma que se borre toda la información del módulo fuente.

Acción: identificar cada hecho por operación/ejecución/acción y entidad, conservando una clave idempotente para reintentos del mismo hecho. Incorporar organización y la semántica de finca correspondiente. Después de definir la identidad, conciliar datos e índices únicos; no añadir un índice único a la clave actual que ya mezcla hechos distintos.

BIT-04 — P1: autoría y revisiones históricas no preservadas

Evidencia: los esquemas tienen `createdAt` y `updatedAt`, pero no `actualizadoPor`, revisiones ni valores anteriores. Los upserts vuelven a escribir `creadoPor`. crearEventosInventario actualiza el evento de nacimiento en cada edición de un animal con fecha de nacimiento, atribuyéndolo al usuario que realiza esa edición. Compras y pesajes también actualizan sus proyecciones con el actor de la edición.

Los productores pueden eliminar la proyección al borrar/anular su fuente: por ejemplo deletePesaje. sincronizarBitacoraTarea elimina eventos previos antes de reconstruirlos, o los deja eliminados al dejar de estar completada la tarea. Por tanto, ni siquiera el ID/createdAt del evento de tarea constituye una revisión estable.

La auditoría HTTP global sí registra usuario, ruta, cuerpo y resultado; no se omite su existencia. Sin embargo, escribe después de finalizar la respuesta, tolera fallos de persistencia y guarda un cuerpo sanitizado/truncado, sin imagen anterior del evento. No garantiza reconstruir el contenido borrado. Su consulta requiere administrador y `auditoriaAvanzada`.

Impacto: el autor mostrado por API puede representar al último sincronizador, no al creador; una corrección no conserva quién cambió qué y por qué. El populate de Usuario tampoco es una instantánea histórica del nombre.

Acción: separar autor original de editor/registrador, registrar revisiones con actor, fecha y motivo, y representar anulaciones sin destruir hechos. Distinguir expresamente fecha del hecho de fecha de carga. Mantener auditoría HTTP como complemento.

BIT-05 — P1: persistencia parcial y reintentos que no reparan la bitácora

Evidencia: createPesaje guarda el pesaje antes de escribir su evento, sin transacción común. En creación reproductiva, incluso cuando el alta/consumo de semen usa transacción, el evento se registra después de completarla.

En completarTarea, la tarea y su clave idempotente se guardan antes de aplicar efectos y sincronizar bitácora. Si ese último paso falla, queda una tarea completada. El reintento con la misma clave retorna por la rama de replay, antes de reparar efectos/eventos. Sin clave, el estado completado también impide repetir normalmente el flujo. Este escenario se infiere del orden del código; no se inyectó un fallo en una base de datos real.

Sanidad implementa compensación al fallar la creación de eventos, lo que reduce el riesgo, pero varias eliminaciones compensatorias tampoco forman una transacción única. Compras, ventas y traslados sí utilizan sesiones en los caminos revisados; el problema no se atribuye indiscriminadamente a todos los productores.

Los helpers además devuelven `null` silenciosamente si faltan campos esenciales. El upsert animal no activa `runValidators`, a diferencia del de camada. Sin índice único de identidad, la idempotencia de upserts concurrentes tampoco queda garantizada por el esquema.

Acción: transacción compartida cuando corresponda o registro durable de eventos pendientes con reintento y reconciliación. No marcar una operación idempotente como finalizada antes de asegurar sus efectos. Fallar explícitamente ante datos incompletos y validar los upserts. Probar fallo entre escritura de fuente/evento y repetición de la misma operación.

BIT-06 — P1: cronología sin validación y estimaciones representadas como hechos

Evidencia: `fecha` solo es Date requerida. POST de animal comprueba existencia, pero no compara nacimiento, muerte o fecha actual; camada ni siquiera comprueba la entidad. Los esquemas aceptaron fechas válidas de 1900 y 2999 en la comprobación aislada. El formato Date inválido sí puede fallar por casting: el problema es la semántica temporal, no la ausencia total de validación de tipos.

En obtenerTipoEventoReproductivo/obtenerFechaEventoReproductivo, un registro sin parto real ni monta, pero con `fechaPartoEstimada`, produce `Diagnostico de gestacion` fechado en el día estimado del parto. Se reprodujo con `2027-01-10`. No se requiere diagnóstico realizado para esa rama. La IA convencional actual normaliza `fechaMonta`; no se le atribuye este error a todas las inseminaciones.

La entrada de lote persiste `datos.fechaEntrada` en PertenenciaLote, pero la llamada al evento no la pasa como `metadata.fecha`; el helper usa la fecha actual. Una entrada retroactiva puede aparecer en otro día en la bitácora.

Orden de presentación: ambas APIs ordenan por `fecha` descendente y luego `createdAt` descendente. Un evento retroactivo aparece en su fecha declarada, no al principio por ser recién cargado. Cambiar `fecha` mediante PUT cambia su posición. La UI no muestra hora ni distingue fecha del hecho/registro. formatearFecha usa la zona del navegador: `2026-10-15T00:00:00Z` aparece como **14/10/2026** en Costa Rica.

Acción: reglas por tipo de hecho, nacimiento/muerte y fecha actual; separar planificación de hechos ejecutados; emitir diagnósticos desde un resultado real y una fecha de diagnóstico; propagar fecha de entrada de lote. Definir una representación consistente para fechas civiles y timestamps antes de ajustar el formato.

4. Comportamiento en Traslados Multi-Finca

BIT-08 — P2: historia consultable de forma condicionada y sin finca visible

El traslado conserva el ID de Animal, cambia su finca, cierra pertenencias de lote y guarda HistorialFincaAnimal y dos EventoAnimal dentro de una transacción. Los eventos anteriores conservan sus fincas originales. Los dos eventos nuevos representan salida e ingreso, no dos traslados independientes.

getEventosPorAnimal primero exige encontrar el animal en la finca activa. Después obtiene las fincas accesibles por membresía y consulta eventos del animal en esas fincas con `omitirAislamientoFinca: true`. **No desactiva el aislamiento organizacional.**

Escenario después de A → B	**Resultado deducido del código**
B activa, usuario autorizado en A y B, ambas activas	Ve eventos anteriores de A y los de B, incluidos ambos lados del traslado.
B activa, usuario autorizado solo en B	No ve eventos de A. Es una restricción de membresía; no debe eliminarse sin una política explícita de acceso al historial clínico.
B activa, A inactiva aunque el usuario tenga acceso a todas	Se excluyen eventos de A: listarFincasAccesibles filtra `estado: Activa` por defecto. La historia clínica queda incompleta sin indicar esa exclusión.
A seleccionada después de mover el animal a B	GET devuelve 404 por no encontrar el animal en la finca activa.

Escenario después de A → B	Resultado deducido del código
Línea de tiempo animal	No popula ni representa el nombre de la finca del evento; los IDs en metadata no se muestran.
Tabla independiente de historial de fincas	Sí presenta finca origen/destino, fecha, peso y motivo. Su endpoint usa ámbito organizacional y no filtra cada fila por la lista de fincas activas. Requiere la función de plan `operacionMultiFinca`.

El historial de traslado visible no sustituye la cronología clínica. Además, ambos endpoints no tienen exactamente la misma política de visibilidad histórica: uno filtra fincas accesibles activas y el otro devuelve la secuencia de traslados del animal dentro de la organización. Debe documentarse y probarse esa diferencia.

Otra consecuencia a cubrir al remediar: al editar un animal ya trasladado, los upserts de inventario se ejecutan bajo la finca B. Pueden crear allí un nuevo evento de nacimiento/compra mientras permanece el de A, porque el plugin incorpora la finca al filtro. La clave de idempotencia debe definir si identifica un hecho del animal durante toda su vida o una proyección por finca; no conviene deduplicar a ciegas los pares legítimos de traslado.

Acción: mostrar finca por evento; distinguir historia parcial de historia vacía; permitir consulta histórica autorizada de fincas inactivas según política; probar A→B con acceso completo/restringido y con A desactivada. Fuentes adicionales: endpoint de historial, tabla de fincas, timeline animal.

5. Matriz de Permisos por Rol

Permisos efectivos de API, sujetos a sesión, membresía y aislamiento de finca/organización. No existe comprobación de propiedad de la nota: el permiso no se limita a notas escritas por el mismo usuario.

Rol	Leer animal	Crear evento animal	Editar animal	Borrar animal	Leer camada	Crear/editar camada	Borrar camada
Administrador	Sí	Sí	Sí	Sí	Sí	Sí	Sí
Encargado	Sí	Sí	No	No	Sí	Sí	No
Veterinario	Sí	Sí	No	No	Sí	Sí	No
Trabajador	Sí	No	No	No	No	No	No
Consulta	Sí	No	No	No	Sí	No	No
Contador	No	No	No	No	No	No	No

En esta tabla, editar/borrar animal significa **evento de bitácora del animal**, no el registro Animal. Fuentes: rutas animal, rutas camada, permisos.

Las rutas de lectura usan `inventario.ver` y `camadas.ver`; la creación animal utiliza `sanidad.gestionar`; edición/borrado animal y borrado camada utilizan `usuarios.gestionar`; creación/edición camada utiliza `camadas.gestionar`. No hay permisos específicos para notas ni filtro por origen automático.

Respuesta a los casos operativos: un trabajador no puede borrar notas de un veterinario/administrador ni crear notas directas. Un veterinario sí puede crear una nota animal por API, pero no editarla después; en camada puede editar eventos de otros autores. Completar tareas asignadas puede generar eventos indirectamente para roles operativos, incluidos trabajador y contador, según tipo de tarea; eso no les concede acceso general a la bitácora.

BIT-09 — P2: desalineación de permisos entre API y formulario

ListaUsuario establece `soloLectura` según gestión de Inventario, concedida a administrador/encargado, o falta de conexión. El formulario manual depende de `!soloLectura`. Por eso el veterinario puede crear por API, pero no ve el formulario. La propiedad `puedeGestionarSanidad` no decide la visibilidad de esta nota.

No se encontró formulario manual dedicado para EventoCamada, ni controles de edición/borrado de eventos individuales en ninguna línea de tiempo. La existencia de funciones de API para actualizar/eliminar eventos animales no implica que estén disponibles en la ficha.

Acción: permisos específicos de lectura/creación/rectificación de notas, compartidos entre backend y frontend; decidir expresamente si Trabajador puede anotar observaciones. Mantener reglas diferenciadas para eventos automáticos.

Visibilidad de contenido a decidir: las lecturas devuelven eventos completos sin ocultar importes o contrapartes según permiso comercial. Compras/ventas escriben precios y proveedor/comprador en metadata y montos en descripción. Trabajador y Veterinario pueden recibirlos por `inventario.ver` aunque no tengan `compras.ver`/`ventas.ver`. Es un comportamiento confirmado, cuya aceptación depende de la política de confidencialidad; si se restringe, debe filtrarse también la descripción, no solo metadata.

6. Plan de Acción Priorizado

Hallazgos funcionales adicionales

BIT-07 — P2: cobertura incompleta de hechos operativos. El cambio de potrero de un lote actualiza animales y crea EventoLote (lote-service), pero no EventoAnimal ni una composición de esa historia en su timeline. Los ajustes de cantidades de camada no generan necesariamente eventos de mortalidad, aunque exista el enum. Alimentación y actividades de protocolos tienen historiales separados; no todos aparecen en la ficha. Definir qué hechos deben ser consultables allí y enlazar/componer sus fuentes, preservando la diferencia entre tarea completada y operación productiva efectivamente registrada.

BIT-10 — P2: presentación insuficiente para trazabilidad. Se popula `creadoPor` pero no se muestra; tampoco editor, hora, finca o revisiones. “Ver detalle” solo ejecuta `window.alert` con módulo e ID (Animales.js), sin abrir la fuente. Las consultas no paginan ni filtran por fecha/tipo; cargan todo el historial. Ante error de consulta también puede aparecer el mensaje “Sin eventos registrados todavía”, confundiendo ausencia con fallo. La interfaz de camada contempla una insignia de origen Sanidad, pero ese origen no pertenece a su enum. Corregir etiquetas, navegación y estados, añadir autor/finca y filtros/paginación; definir adjuntos si son requisito operativo.

BIT-11 — P2: bitácora sin soporte offline. IndexedDB contiene tareas, inventario, potreros, cambios pendientes y metadata, con contexto de organización/finca/usuario; no guarda eventos animales/camadas. La carga de timeline consulta la API sin recuperación desde IndexedDB. Workbox precachea archivos estáticos,

sin caché de API de eventos. Por tanto, una ficha previamente abierta puede conservar datos en memoria durante la sesión, pero no hay garantía de lectura tras recargar sin conexión.

No se pueden encolar notas: offlineSync solo acepta `completar-tarea`, y el formulario se oculta offline. Las observaciones de tareas sí viajan al sincronizar, pero no son notas manuales independientes. El servidor asigna `fechaCompletada = new Date()` al recibirlas; los eventos derivados reflejan la sincronización, no necesariamente la hora real de ejecución en campo. Incorporar caché de eventos y cola de notas con aislamiento de contexto, clave idempotente, fecha de ocurrencia/registro separadas y estado pendiente/conflicto visible.

Prioridades y criterios de cierre

ID	Prioridad	Cambio requerido	Verificación mínima de aceptación
BIT-01	P1	Proteger eventos derivados contra PUT/DELETE directo.	Intentar modificar/borrar venta, pesaje y tratamiento por cada rol autorizado a la ruta; rechazo y fuente/evento conservados.
BIT-02	P1	DTO y asignación segura de autor, tenant, origen y relaciones.	POST/PUT con tenant ajeno, autor ajeno, entidad inexistente y operadores no alteran campos internos ni crean referencias incoherentes.
BIT-03	P1	Identidad por hecho y ejecución, con idempotencia e índice adecuados.	Entrada + salida permanecen como dos hechos; ingreso + retiro de banda y pasos repetidos conservan ambos registros; reintento del mismo hecho no duplica.
BIT-04	P1	Autor original estable, revisiones y anulación trazable.	Editar el nombre del animal no cambia el autor del nacimiento; corregir nota/peso preserva original, actor, motivo y fecha de revisión.
BIT-05	P1	Garantía de persistencia fuente/evento y reparación de reintentos.	Fallo inyectado entre escrituras no deja una operación aparentemente finalizada sin evento; replay de tarea repara o espera sus efectos; concurrencia no duplica.
BIT-06	P1	Fechas de hechos válidas y separación de estimaciones.	Bloquear eventos reales incoherentes con nacimiento/muerte/hoy; parto estimado no crea diagnóstico; entrada retroactiva conserva su fecha; fecha civil no cambia de día en Costa Rica.
BIT-07	P2	Definir cobertura y componer historia de módulos.	Cambio de potrero y ajustes poblacionales requeridos se ven con su fuente; una tarea de pesaje no se presenta como medición inexistente.
BIT-08	P2	Continuidad histórica autorizada y finca visible.	A→B conserva cronología accesible; caso A inactiva resuelto; acceso restringido no filtra información prohibida ni se presenta como historia completa.
BIT-09	P2	Alinear permisos y UI; acordar política de	Veterinario puede usar el formulario permitido por API; trabajador sigue la política acordada; contenido se filtra conforme al permiso definido.

ID	Prioridad	Cambio requerido	Verificación mínima de aceptación
		anotación y contenido comercial.	
BIT-10	P2	Autor, fuente navegable, estados de error y paginación.	Se identifica autor/finca, se abre el registro origen permitido y se distingue carga fallida de historial vacío; consulta paginada mantiene orden.
BIT-11	P2	Lectura y notas offline con sincronización trazable.	Recargar offline conserva historia autorizada; nota pendiente mantiene fecha real y se sincroniza una sola vez; cambiar cuenta/finca no mezcla datos.

Orden recomendado: cerrar BIT-01/02 para asegurar las APIs; resolver BIT-03/04 para estabilizar la identidad y conservación histórica; completar BIT-05/06 y sus pruebas de fallo/fecha; después abordar cobertura, continuidad y uso en campo. Antes de migrar índices o reconstruir eventos, generar un diagnóstico de datos existentes: las claves que ya sobrescribieron hechos no permiten recuperarlos íntegramente desde la bitácora actual.

Estado de entrega: **auditoría completada; remediaciones pendientes**. La suite existente está verde, pero no constituye aceptación de estos hallazgos. La remediación deberá incorporar las pruebas de regresión anteriores y volver a ejecutar la suite completa.